Why Skelet #10 never halts

A guided proof in $I\Delta_0$

Machine behaviour, bounded induction, and the Metamath proof

Explanatory draft, revision 2.

This document explains a formal nonhalting proof for the five-state, two-symbol machine

`1RB0RA_0LC1RA_1RE1LD_1LC0LD_--0RB`.

It starts with the ordinary transition table and the numerical tape encoding. It then explains the two counter routines, the three ways their boundaries fit together, and the single final induction that rules out a halting history.

The intended reader understands first-order logic and induction, but need not know Busy Beaver deciders, bounded arithmetic, or Metamath.

What kind of document is this? It is a detailed mathematical explanation of a machine-checked proof. Definitions are given explicitly; technical deductions use named, already proved lemmas. It does not reproduce every syntax and equality step in the large Metamath file. The final assembly is displayed as a natural-deduction argument, and the exact induction formulas are identified.

Formal source: `idelta0_machines.mm`.

Final theorem:

`skelet17-ladder-skelet10-no-blank-halting-history`.

Theory: the original arithmetic and bounded-induction core; no new machine axioms.

Contents

1 The proof in one picture, without new terminology **2**
1.1 Why the words “in the same history” matter . . . 2
1.2 What is, and is not, being proved . . . 2

2 Reading the machine **3**
2.1 The first three moves . . . 3

3 How a whole tape becomes four numbers **3**
3.1 Deriving the update equations . . . 4
3.2 A numerical transition, worked out . . . 4
3.3 Whole words, and the exact blank prefix . . . 4

4 The arithmetic theory and its allowed induction **4**
4.1 The arithmetic axioms used in this project . . . 5
4.2 The induction schema, including parameters . . . 5
4.3 A small model of the method: division . . . 5

5 What exactly is a coded halting history? **6**
5.1 A complete small coding example . . . 6
5.2 A notation guide . . . 7
5.3 Two consequences of the encoding . . . 7
5.4 Why an ordinary finite halting run has such codes . . . 7

6 Arithmetic predicates for bits and sparse words **8**
6.1 The left-word transform . . . 8

7 The common carry operation **9**
7.1 Why uniqueness is proved, but totality is unnecessary . . . 10

8 What a boundary says **10**

9 The right routine: a carry scan and a marker return **10**
9.1 The actual rank invariant . . . 11
9.2 A complete numerical run . . . 11

10 The left routine: the same carry in a different layout **11**
10.1 The scan phases . . . 12
10.2 The clearing-return invariant . . . 12

11 How local ranks become formal routine theorems **13**

11.1 The halting barrier as natural deduction 13
11.2 Why not simply induct on “has not halted yet”? 14
11.3 Starting a routine in the middle of a history 14

12 Three boundary calculations **15**
12.1 Odd boundary: $n = 4a + 1$ 15
12.2 Even boundary with $n = 4b$ 15
12.3 Even boundary with $n = 8b + 2$ 15
12.4 Why these cases exhaust the invariant 16

13 Putting the pieces together: a worked tour **16**
13.1 What is being carried? 16
13.2 Two right-routine instruction cases, without hidden algebra 17
13.3 A complete left routine, including its return 18
13.4 The first twelve boundaries 19
13.5 Two complete boundary cycles 19

14 The last induction: a greatest boundary index **20**
14.1 Base and successor in detail 20
14.2 Fresh variables in the actual natural deduction 20

15 The final contradiction as a proof table **21**

16 The five induction constructions, collected **22**

17 Finding the corresponding formal proofs **22**
17.1 What verification establishes 23
17.2 What the preceding formal rewrite simplified 23

18 Scope of this draft and further reading **23**

1 The proof in one picture, without new terminology

A Turing machine follows a fixed table, one move at a time. To prove that it never halts, we do not have to predict its entire infinite tape. We find configurations at which two pieces of tape have a useful numerical relationship. Call these configurations *boundaries*.

The proof has three parts.

1. Starting with a blank tape, the machine reaches a boundary after three ordinary moves.
2. In any alleged finite history ending at halt, every boundary is followed by a strictly later boundary in that same history.
3. A nonempty bounded set of indices has a greatest member. Applying this to the boundary indices gives a contradiction: the greatest boundary would have a later boundary.

A Busy Beaver nonhalting decider is a procedure that recognizes some nonhalting patterns and produces a reason for accepting them. Soundness means that accepted machines really do not halt; an unrecognized machine need not halt. Here we prove this particular machine nonhalting directly. No generic decider is assumed sound without proof.

The second part contains the machine-specific work. Its useful pieces are two routines that implement the same numerical carry operation on differently arranged tape words. The routines are not extra instructions of the machine. They are patterns of ordinary instructions whose correctness has been proved.

1.1 Why the words “in the same history” matter

Suppose we merely proved that a boundary has some possible successor boundary somewhere. That would not yet say that the successor belongs to an arbitrary proposed halting computation. The formal proof instead starts with fixed history codes and derives later decoded rows of those very codes. It never changes the computation to a more convenient one.

Likewise, a routine cannot simply assume that the alleged halting history lasts long enough. A halting history might be short. The local routine proofs show that, if its intended output had not occurred, its final row would still be in a nonhalting phase. This contradicts the assumed halt. We will return to this argument in Section 11.

1.2 What is, and is not, being proved

The theorem rules out every supplied finite, bounded, arithmetically coded ordinary computation history that starts blank and ends at halt. In the standard natural numbers, this excludes every actual finite halting computation. It is an individual-machine result; it is not a complete Busy Beaver value or an independence theorem.

The final theorem assumes neither a boundary invariant nor an output of either counter routine. Those facts are established inside the proof and their temporary assumptions are discharged.

2 Reading the machine

There are five working states A through E and two symbols, 0 and 1. An instruction such as `1RB` means: write 1 in the current cell, move one cell right, and enter state B. The two instructions in each state are for reading 0 and reading 1, respectively.

State	Numerical code	Read 0	Read 1
A	0	write 1, right, B	write 0, right, A
B	1	write 0, left, C	write 1, right, A
C	2	write 1, right, E	write 1, left, D
D	3	write 1, left, C	write 0, left, D
E	4	write 1, right, Z	write 0, right, B

State Z, numerically 5, is halt and has no outgoing instruction. The printed `--` is completed by `1RZ`. Thus the final write and move of a halting instruction are part of the transition relation. The proof shows that this halting transition is never reached from blank.

2.1 The first three moves

The initial state is A and every tape cell contains 0.

1. A reads 0, writes 1 and moves right to B. The new cell is blank.
2. B reads 0, writes 0 and moves left to C. The head is back on the 1 written in the first move.
3. C reads 1, writes 1 and moves left to D. The newly scanned cell is blank; the nearest cell on its right contains 1.

This last configuration will be the first boundary.

> **Reading note.** The simplified final proof uses these three moves directly. A previous version invoked the entire right-counter routine with input 0 to obtain the first boundary. The routine theorem is still useful later, but its start-up application is unnecessary.

3 How a whole tape becomes four numbers

A configuration is a quadruple

$$c = (q, L, s, R).$$

Here q is the state code and s is the scanned symbol. The integer L encodes the half-tape to the left of the head and R encodes the half-tape to the right. In either half, the nearest bit is the least significant bit. The scanned cell belongs to neither L nor R.

For example, $L = 5$ has binary expansion 101. Reading leftwards away from the head, the next three cells contain $1, 0, 1$; the remaining cells are 0. The same integer used as R describes $1, 0, 1$ while reading rightwards away from the head. The direction is determined by which coordinate is being described.

The quadruple $(3, 5, 0, 2)$ therefore means: state D; scanned symbol 0; left half with nearest-first bits $1, 0, 1$; right half with nearest-first bits $0, 1$. The formal proof does not construct a binary expansion for every arithmetic value. It reasons with the update equations and bounded predicates below, keeping its arithmetic requirements explicit.

3.1 Deriving the update equations

Appending a nearest bit e to an encoded half-tape x produces $2x + e$. Removing its nearest bit gives an equation $x = 2x' + s'$, with $s' < 2$.

A move right, writing e, therefore satisfies

$$L' = 2L + e, \qquad R = 2R' + s', \qquad s' = 0 \ \vee \ s' = 1. \tag{1}$$

The written cell becomes the nearest cell on the left. The old nearest right cell becomes the scanned cell. A move left instead satisfies

$$R' = 2R + e, \qquad L = 2L' + s', \qquad s' = 0 \ \vee \ s' = 1. \tag{2}$$

The source and target state equations, and the source read-bit equation, complete each instruction clause. $\mathrm{Step}(c, c')$ is the disjunction of the ten such clauses. These are equations defining the ordinary machine, not equations silently supplemented with an algorithm.

3.2 A numerical transition, worked out

Consider D reading 0 in $(3, 5, 0, 2)$. It writes 1 and moves left to C. Equation (2) gives

$$R' = 2 \cdot 2 + 1 = 5, \qquad 5 = 2L' + s', \qquad s' < 2.$$

Quotient/remainder uniqueness forces $L' = 2$ and $s' = 1$. The successor is $(2, 2, 1, 5)$. Every number has a role; none is a hidden tape annotation.

3.3 Whole words, and the exact blank prefix

We use the term abbreviations

$$\mathrm{WL}(c) = 2L + s, \qquad \mathrm{WR}(c) = 2R + s.$$

Each includes the scanned cell. They are not extra coordinates. In the first three moves the quadruples are exactly

Index	State	L	s	R	Reason for the next move
0	A	0	0	0	A0
1	B	1	0	0	B0
2	C	0	1	0	C1
3	D	0	0	1	the first boundary

The simplified proof derives three fixed-source instruction lemmas, then applies them to the actual decoded history rows. It does not assume that this table was supplied as part of the input history.

4 The arithmetic theory and its allowed induction

The language contains $0, 1, +, \cdot, <$ and equality. Numerals such as 10 and 32 abbreviate sums of 1. A formula is *bounded*, or Δ_0, when every quantifier has an explicit term bound. The notation means

$$\exists x < t\, A \quad \text{abbreviates} \quad \exists x\, (x < t \wedge A),$$
$$\forall x < t\, A \quad \text{abbreviates} \quad \forall x\, (x < t \to A).$$

The bounding term does not contain the quantified variable. Bounds may depend on parameters and on surrounding quantified variables.

4.1 The arithmetic axioms used in this project

All instances of the following axioms are available, together with classical first-order natural deduction and equality substitution:

$$\begin{array}{ccc} (x+y)+z = x+(y+z), & x+y = y+x, & x+0 = x, \\ (xy)z = x(yz), & xy = yx, & x\cdot 1 = x, \\ x\cdot 0 = 0, & x(y+z) = xy+xz. & \end{array}$$

The order axioms are

$$\begin{array}{c} \neg(x<x), \qquad (x<y \wedge y<z) \to x<z, \\ x<y \ \vee \ x=y \ \vee \ y<x, \qquad \neg(x<0), \\ x<y \to x+z<y+z, \\ (x<y \wedge 0<z) \to xz<yz, \\ x<y \quad \longleftrightarrow \quad \exists z\,[y=(x+z)+1]. \end{array}$$

The last line expresses discreteness: a strict increase has at least one successor step. Its quantified variable is fresh. These are the project's specified PA^- axioms; $I\Delta_0$ adds bounded induction.

4.2 The induction schema, including parameters

For every Δ_0 formula $\varphi(i;\bar{a})$, we may use

$$(\varphi(0;\bar{a}) \wedge \forall i\,[\varphi(i;\bar{a}) \to \varphi(i+1;\bar{a})]) \ \to \ \forall i\,\varphi(i;\bar{a}). \tag{3}$$

Here i is the variable being inducted on. The tuple $\bar{a}$ contains parameters held fixed throughout the induction: for example the history codes, its coordinate bound H, and its final index N.

The quantifier $\forall i$ outside φ is unbounded. This is permitted: the restriction is that the *formula inducted on* is bounded. An implication containing bounded quantifiers is still bounded. By contrast, $\forall y\,\exists z\,A(i,y,z)$ would not be a permitted induction formula without further restrictions on its quantifiers.

The proof does not add exponentiation, arbitrary finite-sequence existence, or a rule saying that every Turing computation has a code. Its bounded graph definitions are expanded formulas of the stated language.

4.3 A small model of the method: division

For fixed positive d, the division proof inducts on

$$D(i;d) \ := \ \exists q < i+1\,\exists r < d\ [i = dq + r].$$

At 0 choose $q = r = 0$. At the successor, either $r+1 < d$, and retain q, or $r+1 = d$, and replace (q,r) by $(q+1,0)$. The bounds are checked in each case. This is one allowed bounded-induction construction.

Uniqueness of quotient and remainder, once they are given, uses order and arithmetic without a further induction. This is why the tape equations can determine a popped bit without assuming a binary-expansion function.

5 What exactly is a coded halting history?

Fix natural-number parameters

$$H, N, o, B_0, C_0, B_1, C_1, B_2, C_2, B_3, C_3.$$

N is the final local row index, so there are N transitions and $N+1$ rows. H bounds all row coordinates and all quotient witnesses used below. o is an offset; setting $o = 0$ gives the ordinary start of a history. The letters B_j, C_j are numbers used for coding, not machine states.

Define

$$\beta_j(k, x) \iff \exists u < H \ \big[B_j = u(1 + (k+1)C_j) + x \ \wedge \ x < 1 + (k+1)C_j\big]. \tag{4}$$

This is a quotient-and-remainder equation: x is the remainder of B_j modulo $1 + (k+1)C_j$. The variable u is a quotient witness, not a tape cell or step counter. We then define

$$\mathrm{At}_o(i, c) \iff \bigwedge_{j=0}^{3} \beta_j(o + i, c_j).$$

Coordinates $j = 0, 1, 2, 3$ respectively mean state, left half, scanned bit, and right half.

Write $c < H$ for four separate coordinate inequalities, and use the same abbreviation for four bounded quantifiers. The history conditions are

$$\begin{aligned}
\mathrm{Total}_o(N) &\iff \forall i < N+1 \ \exists c < H \ \ \mathrm{At}_o(i, c),\\
\mathrm{Steps}_o(N) &\iff \forall i < N \ \forall c < H \ \forall d < H \ [(\mathrm{At}_o(i, c) \wedge \mathrm{At}_o(i+1, d)) \to \mathrm{Step}(c, d)],\\
\mathrm{Blank}_o &\iff \forall c < H \ [\mathrm{At}_o(0, c) \to c = (0, 0, 0, 0)].
\end{aligned}$$

The tuple equality is four ordinary equalities. Total ensures an actual decoded initial row, so Blank is not used vacuously to manufacture one.

For a specified final row h, the final theorem is

$$\neg\big[\mathrm{Total}_o(N) \wedge \mathrm{Steps}_o(N) \wedge \mathrm{Blank}_o \wedge h < H \wedge \mathrm{At}_o(N, h) \wedge h_0 = 5\big]. \tag{5}$$

A halt on the last transition is recorded in the final row h, including its write and move; the final transition is not omitted from Steps. Its universal closure over the displayed numerical parameters is the nonhalting assertion. In particular, $o = 0$ is the usual blank-start instance. No boundary or counter certificate occurs in (5).

5.1 A complete small coding example

The three-step prefix from Section 3 can be encoded with $C_0 = C_1 = C_2 = C_3 = 12$, $o = 0$, $N = 3$, and

$$(B_0, B_1, B_2, B_3) = (49101, 94276, 79625, 324675).$$

The four divisors are $13, 25, 37, 49$. Choose $H = 600000$; every coordinate and every quotient in the following table is smaller than H.

i	divisor	B_0 quotient/remainder	B_1 q/r	B_2 q/r	B_3 q/r
0	13	3777 / 0	7252 / 0	6125 / 0	24975 / 0
1	25	1964 / 1	3771 / 1	3185 / 0	12987 / 0
2	37	1327 / 2	2548 / 0	2152 / 1	8775 / 0
3	49	1002 / 3	1924 / 0	1625 / 0	6626 / 1

For example $49101 = 1327 \cdot 37 + 2$ decodes state C at index 2. The remainder columns give exactly the four configurations in the earlier table. These constants are a checked example, not additional axioms or bounds required by the universal theorem. $C = 12$ gives convenient, pairwise coprime moduli for this example; the theorem does not require every supplied code to have that property.

5.2 A notation guide

Symbol	Meaning in the global proof
N	Final history index; there are N transitions and $N + 1$ rows.
i, j, k, e	History indices. k is often the current boundary index and e its later output.
H	A fixed bound on decoded coordinates and beta quotient witnesses. It is not a running-time bound.
o	History offset; set $o = 0$ for the ordinary blank start.
c, d, h	Configuration quadruples; h denotes the proposed final halt row.
n	An input interpreted at a boundary. It is distinct from the history length N.
w, ℓ, r	Local routine parameters: input word, preserved left word, or preserved right tail.
p, a, z	Local carry witnesses in J. They are eliminated bounded variables.
ρ	A positive rank used in a routine proof, not a coordinate stored in the history.
$\bar{a}$	An arbitrary tuple of parameters in the general induction schema, not a single carry witness.

5.3 Two consequences of the encoding

Quotient/remainder uniqueness proves that a coordinate has at most one value at a fixed index. Consequently two decoded quadruples at an index are equal. This does not require induction on the history.

Inside an alleged halting history, a nonhalting decoded row at $i < N + 1$ must have $i < N$. If $i = N$, uniqueness would identify it with the final halt row. Total then supplies a successor at $i + 1$, and Steps supplies its ordinary transition. This is the *advance-before-halt* lemma. It is the bridge from an informal finite calculation to actual rows of an arbitrarily supplied history.

5.4 Why an ordinary finite halting run has such codes

There is a separate adequacy question: could a real finite halting run escape the arithmetic statement simply because it has no codes? In the standard natural numbers, it cannot. Here is the usual finite-list coding argument, given in ordinary mathematics, not as an additional theorem assumed inside $I\Delta_0$.

Given a finite run with indices $0, \ldots, N$, choose a positive integer C larger than all its coordinates and divisible by every integer from 1 through N. A sufficiently large multiple of $N!$ works; for $N = 0$ there is no divisibility requirement. Put $m_i = 1 + (i + 1)C$.

These moduli are pairwise coprime. A common divisor g of m_i, m_j divides $(j - i)C$ and is coprime to C. Hence it divides $j - i$. For $i \neq j$, the integer $|j - i|$ divides the chosen C, so g also divides C; therefore $g = 1$.

The finite Chinese remainder theorem gives, for each coordinate sequence, a number B_j with precisely its prescribed values as remainders modulo $m_0, \ldots, m_N$. Those values are smaller than every relevant modulus because C was chosen larger than them. Choose H larger than the

coordinates and the finitely many quotient witnesses. With $C_j = C$ and $o = 0$, the run satisfies the displayed beta-history conditions.

Thus any actual finite halting run would give an ordinary numerical instance contradicted by (5). Factorials and Chinese-remainder coding are used only in this explanation of the standard meaning of the encoding. The formal nonhalting proof neither adds them as axioms nor needs to prove unrestricted history-code totality in $I\Delta_0$.

6 Arithmetic predicates for bits and sparse words

The proof uses bit language for intuition, but no bit-extraction function is added. Define

$$\begin{aligned}
\mathrm{Div}(d,p) &:\Longleftrightarrow \exists e < p+1\,[de = p],\\
\mathrm{Even}(d) &:\Longleftrightarrow \exists h < d+1\,[d = 2h],\\
\mathrm{Pow}(p) &:\Longleftrightarrow 0 < p \wedge \forall d < p+1\,[\mathrm{Div}(d,p) \to (d = 1 \vee \mathrm{Even}(d))],\\
\mathrm{Bit}(x,p) &:\Longleftrightarrow \exists q < x+1\,\exists r < p\,[x = q(2p) + p + r].
\end{aligned}$$

In the standard natural numbers, $\mathrm{Pow}(p)$ identifies positive powers of 2. If p is such a place value, $\mathrm{Bit}(x,p)$ says its bit in x is 1. The proof uses these bounded formulas and their arithmetic lemmas, not an unbounded existential exponent.

No adjacent 1s means

$$\mathrm{Sparse}(x) :\Longleftrightarrow \forall p < x+1\,[\mathrm{Pow}(p) \to \forall q < x+1\,\forall r < p\; \neg(x = (4p)q + 3p + r)].$$

The term $3p = 2p + p$ accounts for two adjacent 1 bits at places p and $2p$. Examples of sparse words are $0, 1, 2, 4, 5, 8, 9, 10$; 3 and 6 are not sparse. The bounds $p, q < x + 1$ are sufficient because a forbidden positive place and its quotient would already be bounded by x.

Three repeatedly used deductions are

$$\begin{gathered}
\mathrm{Sparse}(2x+d) \wedge d < 2 \;\to\; \mathrm{Sparse}(x),\\
\mathrm{Sparse}(2x+1) \;\to\; \mathrm{Even}(x),\\
\mathrm{Sparse}(x) \;\to\; \mathrm{Sparse}(2x), \qquad \mathrm{Sparse}(x) \;\to\; \mathrm{Sparse}(4x+1).
\end{gathered}$$

The middle deduction expresses the crucial fact that a nearest 1 must have a neighbouring 0. Its formal proof divides x by 2; if the remainder were 1, $2x + 1$ would have forbidden low bits 11.

6.1 The left-word transform

The left routine uses a spaced-out representation. First define

$$\begin{aligned}
\mathrm{Dil}(x,z) :\Longleftrightarrow\; &\forall p < x+1\,[(\mathrm{Pow}(p) \wedge \mathrm{Bit}(x,p)) \to \mathrm{Bit}(z,p^2)]\\
&\wedge \forall q < z+1\,[(\mathrm{Pow}(q) \wedge \mathrm{Bit}(z,q)) \to\\
&\qquad \exists p < x+1\,(\mathrm{Pow}(p) \wedge q = p^2 \wedge \mathrm{Bit}(x,p))].
\end{aligned}$$

Here p^2 abbreviates $p \cdot p$. The two directions ensure both that each input 1 contributes the intended output 1 and that the output has no other 1s. Now put

$$\mathrm{T}(x,t) :\Longleftrightarrow \exists z < t+1\,[t = 10z \wedge \mathrm{Dil}(x,z)].$$

Informally z moves a bit at 2^i to 4^i; multiplying by 10 produces the tape layout used by the left routine. This explanation uses exponents only in the metatheory. The definition uses multiplication and bounded quantifiers.

Input x	binary input	spread value z	transform $t = 10z$
0	0	0	0
1	1	1	10
2	10	4	40
4	100	16	160
5	101	17	170
8	1000	64	640

The exact constructors and bounded inverses used later are

$$\begin{aligned} \mathrm{T}(x,t) &\to \mathrm{T}(2x,4t),\\ \mathrm{T}(x,t) &\to \mathrm{T}(4x+1,16t+10),\\ \mathrm{T}(2x,z) &\to \exists t<z+1\,[z=4t\wedge \mathrm{T}(x,t)],\\ \mathrm{T}(4x+1,z) &\to \exists t<z+1\,[z=16t+10\wedge \mathrm{T}(x,t)]. \end{aligned}$$

The factor 10 is binary 1010. Consequently an even input has the encoded low bits 00, while a sparse odd input $4x+1$ has encoded low bits 1010. These are exactly the patterns consumed by the D/C scan: the first leads into the exit path and the second through four carry moves. The graph T describes a relationship between ordinary tape values; it does not change the quadruple encoding of the machine.

These inverses recover a bounded tail from an existing graph assertion. They do not assume that every input has an output or that all outputs are unique. No transform-totality or transform-uniqueness premise is needed in the nonhalting proof. Starting with the concrete fact $\mathrm{T}(0,0)$, the proof only needs to preserve existing graph assertions and recover bounded tails from them. It never has to find a transformed output for an arbitrary input unrelated to the supplied history.

7 The common carry operation

Both routines implement the bounded relation

$$\begin{aligned} \mathrm{J}(w,v) \;:\Longleftrightarrow\; \exists p<3w+2\,\exists a<w+1\,\exists z<w+1:\\ \mathrm{Pow}(p)\wedge p=3a+1\wedge w=p(2z)+a\wedge v=p(4z+1). \end{aligned} \tag{6}$$

The letter a here is the consumed low suffix, and z is a remaining tail. They are local bound variables, not the input parameter a that will be used in a later case calculation.

For standard binary words, the relation follows the recursion

$$\mathrm{J}(2x,4x+1), \qquad \mathrm{J}(x,v)\to \mathrm{J}(4x+1,4v).$$

Thus an even word is the base case. A low block 01, written in the usual most-significant-first binary notation, triggers another carry. When reading tape bits nearest-first, that block is 10.

w	v	p	consumed suffix a	tail z
0	1	1	0	0
1	4	4	1	0
2	5	1	0	1
4	9	1	0	2
5	16	16	5	0

For $w=5$, check $16=3\cdot 5+1$, $5=16\cdot(2\cdot 0)+5$, and $16=16(4\cdot 0+1)$. This is not ordinary successor: $\mathrm{J}(1,4)$, not $\mathrm{J}(1,2)$.

7.1 Why uniqueness is proved, but totality is unnecessary

The routine contracts can produce different existential witnesses for what should be the same carry output. We therefore prove

$$\mathrm{Sparse}(w) \wedge \mathrm{J}(w, v) \wedge \mathrm{J}(w, v') \;\to\; v = v'.$$

The even case is forced by the inverse $\mathrm{J}(2x, v) \to v = 4x + 1$. In the odd sparse case, write $w = 4x + 1$. The carry inverse produces smaller outputs u, u' with $v = 4u$, $v' = 4u'$, and $\mathrm{J}(x, u), \mathrm{J}(x, u')$. Strong induction on the input then gives $u = u'$.

The actual allowed induction formula, for a fixed output bound U, is

$$\forall x < k + 1\ \forall v < U\ \forall v' < U\,[\mathrm{Sparse}(x) \wedge \mathrm{J}(x, v) \wedge \mathrm{J}(x, v') \to v = v']. \tag{7}$$

Every quantifier is bounded. The smaller outputs remain below U because their inverse witnesses are bounded by the old outputs. To obtain the unrestricted displayed implication for particular v, v', instantiate $U = v + v' + 1$. No assertion $\forall w \exists v\, \mathrm{J}(w, v)$ is used to do this.

8 What a boundary says

A boundary is a D configuration satisfying $s < 2$, $\mathrm{Sparse}(n)$ and $\mathrm{Sparse}(R)$, together with one of two bounded clauses:

$$\begin{aligned} E(n, c) &: \exists a < n + 1\ [n = 2a \wedge \mathrm{T}(a, \mathrm{WL}(c)) \wedge \mathrm{J}(a, R)], \\ O(n, c) &: \exists a < n + 1\, \exists t < \mathrm{WL}(c) + 1\, \exists v < R + 1 : \\ &\qquad n = 4a + 1 \wedge \mathrm{WL}(c) = 4t + 2 \wedge \mathrm{T}(a, t) \wedge R = 2v \wedge \mathrm{J}(a, v). \end{aligned}$$

Thus

$$\mathrm{Boundary}(n, c) \;:\Longleftrightarrow\; q = 3 \wedge s < 2 \wedge \mathrm{Sparse}(n) \wedge \mathrm{Sparse}(R) \wedge (E(n, c) \vee O(n, c)).$$

The parameter n is an interpreted counter input. It is not the elapsed number of steps and is not a fifth tape coordinate.

The first boundary is $\mathrm{Boundary}(0, (3, 0, 0, 1))$. Choose $a = 0$ in the even clause, use $\mathrm{T}(0, 0)$ and $\mathrm{J}(0, 1)$, and note that 0 and 1 are sparse. The witness bound is $a < 1$.

The two clauses accommodate different alignments of the head and low bits. They let the same invariant describe the boundary after different carry paths. The extra numbers a, t, v are proved to exist and are bounded in the formula; they are not supplied with a computation.

9 The right routine: a carry scan and a marker return

Fix arbitrary input w and an arbitrary preserved left word ℓ. The entry and output contracts are

$$\begin{aligned} \text{entry: } & q = B, \quad L = 2\ell + 1, \quad \mathrm{WR}(c) = w, \quad s < 2, \quad \mathrm{Sparse}(w), \\ \text{output: } & q = D, \quad \mathrm{WL}(c) = \ell, \quad \mathrm{J}(w, R), \quad \mathrm{Sparse}(R). \end{aligned}$$

The nearest left bit 1 is a marker. The B/A part consumes carry blocks; B0 and C1 start the return; D1 moves back across the marker.

9.1 The actual rank invariant

Put $K = 16(w+1)$. A phase certificate $\mathrm{Rank}_R(c,\rho)$ is a bounded disjunction of the following alternatives. In the B,A,C alternatives, $p < 3w+2$, $a < w+1$, $\mathrm{Pow}(p)$, and $p = 3a+1$ are common data.

Phase	Tape and arithmetic conditions	Rank ρ
B	$L+1 = 2p(\ell+1)$; $w = p(2R+s)+a$; $s<2$; $\mathrm{Sparse}(2R+s)$	$K+4(2R+s)+2$
A	$s=0$; $L+1 = 4p(\ell+1)$; $w = 4pR+4a+1$; $\mathrm{Sparse}(R)$	$K+16R+5$
C	$s=1$; $L+1 = p(\ell+1)$; $w = pR+a$; $\mathrm{Even}(R)$; $\mathrm{Sparse}(R)$	$K+4R+1$
D	Some $\eta < 3w+2$ has $\mathrm{Pow}(\eta)$, $\mathrm{WL}(c)+1 = \eta(\ell+1)$, $\mathrm{J}(w,\eta R)$; $s<2$; $\mathrm{Sparse}(R)$	4η

The D return marker η is not required to equal the scale occurring inside the independent existential assertion $\mathrm{J}(w,\eta R)$.

At entry use $p=1, a=0$ and obtain rank

$$R_0 = 16(w+1)+4w+2 = 20w+18.$$

The local theorem checks every ordinary instruction alternative. B1 gives A0 because sparsity forces the next bit to be 0. A0 updates p to $4p$ and a to $4a+1$. B0 pops the marker bit and gives C1. C1 supplies the witnesses for the carry graph. Until output is reached, D1 halves the remaining marker and preserves the encoded final output. Every unfinished transition has a strictly smaller positive rank.

9.2 A complete numerical run

Take $w=5$ and $\ell=2$. The input is $(B,5,1,2)$ and the carry output should be 16.

Move	State	L	s	R	constructed rank
0	B	5	1	2	118
1	A	11	0	1	117
2	B	23	1	0	102
3	A	47	0	0	101
4	B	95	0	0	98
5	C	47	1	0	97
6	D	23	1	1	64
7	D	11	1	2	32
8	D	5	1	4	16
9	D	2	1	8	8
10	D	1	0	16	4

At the final row $\mathrm{WL} = 2 = \ell$ and $\mathrm{J}(5,16)$. The routine theorem stops describing the machine there; the machine itself continues. The rank bound 118 is deliberately loose. It is not a claim that this particular run takes 118 steps.

10 The left routine: the same carry in a different layout

Fix input w and preserved right tail r. The contracts are

$$\begin{aligned} \text{entry: } & q = D, \quad s<2, \quad \mathrm{Sparse}(w), \quad \mathrm{T}(w,\mathrm{WL}(c)), \quad R = 4r+3, \\ \text{output: } & q = A, \quad \mathrm{WR}(c) = r, \\ & \exists V < 3w+2\,[\mathrm{J}(w,V) \wedge \mathrm{T}(V,L) \wedge \mathrm{Sparse}(V)]. \end{aligned}$$

The low right bits 11, expressed by $R = 4r + 3$, provide the return layout. The left tape stores a transformed input. The routine returns a transformed carry output in its left half.

10.1 The scan phases

Set $S = 3w + 2$ and $K = 8S$. The shared bounded scan data are

$$p < S, \quad a < w + 1, \quad u < w + 1, \quad \mathrm{Pow}(p), \quad p = 3a + 1, \quad w = pu + a, \quad \mathrm{Sparse}(u).$$

Put $F = p^2(r + 1)$. In the following table, whenever t and b occur they satisfy $\mathrm{Sparse}(b)$ and $\mathrm{T}(b, t)$, with bounds $b < w + 1$ and $t < L + 1$. The rank is $K + 8u$ plus the last column.

Phase	State/read	Left information	$R + 1$	Restriction	offset
Search	D / $s < 2$	$\mathrm{T}(u, 2L + s)$	$4F$	none	8
Exit 1	C / 0	$L = t$	$8F$	$u = 2b$	7
Exit 2	E / 1	$L = 2t + 1$	$4F$	$u = 2b$	6
Exit 3	B / 1	$L = 4t + 2$	$2F$	$u = 2b$	5
Exit 4	A / 1	$L = 8t + 5$	F	$u = 2b$	4
Carry 1	C / 1	$L = 4t + 2$	$8F$	$u = 4b + 1$	7
Carry 2	D / 0	$L = 2t + 1$	$16F$	$u = 4b + 1$	6
Carry 3	C / 1	$L = t$	$32F$	$u = 4b + 1$	5

At Search, bounded division and the sparse-tail lemma distinguish $u = 2b$ from $u = 4b + 1$. The inverse transform lemmas recover t with the correct tape equation. The carry path returns to Search with

$$p' = 4p, \qquad a' = 4a + 1, \qquad u' = b.$$

The equations are preserved and the rank decreases because $b < 4b + 1$.

The exit path constructs $V = p(4b + 1)$ and the witnesses for $\mathrm{J}(w, V)$. The arithmetic equations give $V < 3w + 2$. It then enters a clearing return. In particular E is scanned only on 1 in the exit path: the halting E0 instruction is not hidden inside the routine.

10.2 The clearing-return invariant

This is part of the same complete left-routine induction. It does not require an additional history induction in the final proof. Write $P_0 = 3w + 1$ and fix the carry output V. Both clearing phases have $q = A$, $s < 2$, $p < P_0 + 1$, $u < V + 1$, $\mathrm{Pow}(p)$, and $\mathrm{Sparse}(u)$. Their remaining conditions are

$$\begin{aligned} \text{main:} \quad & \mathrm{T}(u, L), \quad pu = V, \quad \mathrm{WR}(c) + 1 = p^2(r + 1), \quad \rho = 2p, \\ \text{intermediate:} \quad & \exists t < L + 1 : \ \mathrm{T}(u, t), \quad L = 2t, \quad 2pu = V, \\ & \mathrm{WR}(c) + 1 = 2p^2(r + 1), \quad \rho = 2p + 1. \end{aligned}$$

Two A1 moves reduce a larger main scale to its half. Simultaneously the transform constructor restores the new encoded left word. At main scale 1, the equations imply $\mathrm{WR}(c) = r$ and $\mathrm{T}(V, L)$, which is the desired output. The full left certificate is the disjunction of a scan phase and a bounded $V < S$ with $\mathrm{J}(w, V)$ and a clearing phase.

The initial full rank is

$$L_0 = K + 8w + 8 = 32w + 24.$$

All unfinished steps preserve a positive certificate with strictly smaller rank. For $r = 0$, inputs $w = 0, 1, 5$ start with whole-left words $0, 10, 170$ and return carry outputs $1, 4, 16$ after respectively $5, 13, 21$ ordinary moves. These are examples, not the basis of the universal proof.

11 How local ranks become formal routine theorems

For either routine, let R_0 denote its initial rank and let $\mathrm{End}(c)$ be its stated output condition. Temporarily assume that all decoded rows avoid End. For fixed history parameters, the exact shape of the induction formula is

$$\Phi(i): \quad i < N+1 \;\rightarrow\; \forall c < H\,[\mathrm{At}_o(i,c) \rightarrow \exists \rho < R_0 + 1\,(\mathrm{Rank}(c,\rho) \wedge i + \rho < R_0 + 1)]\,. \tag{8}$$

The left formula uses $R_0 = 32w + 24$ and Rank_L; the right formula uses $R_0 = 20w + 18$ and Rank_R. These are two distinct bounded-induction constructions, with fixed numerical parameters H, N, o, w and the codes.

The base obtains the entry row's certificate at rank R_0. For the successor, Total supplies a predecessor and Steps its transition. The local theorem provides $\rho' < \rho$. Discreteness turns this into the clock inequality

$$(i+1) + \rho' < R_0 + 1.$$

The witnesses stay bounded by $R_0 + 1$, so (8) is Δ_0. All quantifiers inside the rank relation are also bounded, as the phase descriptions specify.

If the history is sufficiently long, positive rank contradicts its length. More importantly here, if the final row is halt, its rank certificate contradicts its state: every certified phase is nonhalting. Negating the assumed avoidance yields a bounded existential output row. This is the routine's *halting barrier*. It works even for a proposed history shorter than the coarse rank bound.

11.1 The halting barrier as natural deduction

Let $\mathcal{R}$ abbreviate a routine's history premises: total rows, ordinary transitions, its entry at index 0, and a decoded final row h in state 5. Define

$$E := \exists j < N + 1\, \exists c < H\,[\mathrm{At}_o(j,c) \wedge \mathrm{End}(c)].$$

This is bounded; c abbreviates four separately bounded coordinates. The following argument uses the invariant theorem already proved with the displayed bounded induction, and introduces no further induction.

Line	Statement	Justification
1	Assume $\mathcal{R}$.	Assumption for implication introduction.
2	Assume $\neg E$.	Assumption for a classical contradiction argument.
3	$\forall j < N+1\, \forall c < H\, [\mathrm{At}_o(j,c) \to \neg\, \mathrm{End}(c)]$.	First-order quantifier and propositional laws, 2.
4	$\Phi(i)$ holds for every i.	The proved routine invariant, using the history premises and 3.
5	$h < H$, $\mathrm{At}_o(N,h)$ and $h_0 = 5$.	Conjunction elimination, 1.
6	$N < N+1$.	Ordered-arithmetic successor lemma.
7	$\exists \rho < R_0 + 1\, [\mathrm{Rank}(h,\rho) \wedge N + \rho < R_0 + 1]$.	Universal and implication eliminations in 4, using 5,6.
8	Fix such a fresh ρ; $\mathrm{Rank}(h,\rho)$.	Existential and conjunction eliminations.
9	$h_0 \neq 5$.	Every disjunct of the rank relation has a working state.
10	Contradiction.	5 and 9; discharge the fresh witness.
11	$\neg\neg E$, hence E.	Negation introduction discharging 2; classical double-negation elimination.
12	$\mathcal{R} \to E$.	Implication introduction discharging 1.

This is a natural-deduction explanation using the named invariant and arithmetic lemmas, not a printout of their expanded kernel proofs. Line 9 is the reason a short proposed halting history is excluded too: we use its final state, not a lower bound on its duration. No premise says that the history allows the routine time to finish. Rather, the argument proves that any history ending at halt has already visited the required output.

11.2 Why not simply induct on "has not halted yet"?

That phrase conceals the difficult part: what numerical data describe the configuration at an arbitrary time? Proving that suitable history codes exist for every time would introduce unbounded existential code and bound variables. Such a statement is not automatically an allowed Δ_0 induction formula.

Instead, the proof assumes one proposed finite halting history and fixes its codes, H and N as parameters. The guard $i < N + 1$ makes the induction vacuous beyond that window. Inside the window, totality provides bounded rows, and the invariant provides bounded rank witnesses. The resulting formula is genuinely bounded. This proves that no proposed halting history works, without proving a stronger computation-totality theorem.

11.3 Starting a routine in the middle of a history

If a known entry occurs at $k < N + 1$, ordered arithmetic supplies a bounded remaining length $m < N + 1$ satisfying $k + m = N$. The same codes at offset $o + k$ satisfy the suffix totality and transition conditions. This is a change of indices, not a new encoding.

The embedded barrier constructs a later output index $e < N + 1$ and a bounded decoded row at that index. Row uniqueness identifies the entry with any decoded row at its starting index. Its

output differs in state from its entry, giving strict progress. The output's scanned-bit bound is derived from the ordinary incoming transition.

12 Three boundary calculations

Every calculation below takes place in a fixed alleged halting history. Each displayed move is obtained using advance-before-halt, and routine outputs are obtained using the embedded barriers. Consequently the later boundary belongs to the original history, even if the proposed history was initially supposed to halt during the calculation.

12.1 Odd boundary: $n = 4a + 1$

The odd clause supplies $\mathrm{WL} = 4t + 2$, $\mathrm{T}(a, t)$, $R = 2v$, and $\mathrm{J}(a, v)$. Since the boundary also gives $\mathrm{Sparse}(4a + 1)$, two sparse pop deductions give $\mathrm{Sparse}(a)$, as required by the left entry. Two moves D0,C1 give a D row with

$$\mathrm{WL} = t, \qquad R = 4R_{\mathrm{old}} + 3.$$

This is a left entry with input a and tail R_{old}. Its actual output has state A and $\mathrm{WR} = R_{\mathrm{old}} = 2v$. It supplies an output V with $\mathrm{J}(a, V)$ and $\mathrm{T}(V, L)$. Sparse carry uniqueness identifies $V = v$.

One ordinary A0 move enters the right routine with input v and left target equal to this A-row's L. Its D output therefore satisfies $\mathrm{T}(v, \mathrm{WL})$, $\mathrm{J}(v, R)$ and $\mathrm{Sparse}(R)$. This is the even boundary with input $2v = R_{\mathrm{old}}$.

12.2 Even boundary with $n = 4b$

Its even witness is $a = 2b$, and two sparse pops give $\mathrm{Sparse}(b)$. Carry inversion gives $R_{\mathrm{old}} = 4b+1$. Transform inversion gives $\mathrm{WL} = 4t$ and $\mathrm{T}(b, t)$, so the source scanned bit is 0 and $L = 2t$. The actual four-move prefix is

Point	State	L	s	R
source	D	$2t$	0	$4b + 1$
after D0	C	t	0	$8b + 3$
after C0	E	$2t + 1$	1	$4b + 1$
after E1	B	$4t + 2$	1	$2b$
after B1	A	$8t + 5$	0	b

An A0 move enters the right routine with input b and left target $8t + 5$. Its D output has $\mathrm{WL} = 8t + 5$ and $\mathrm{J}(b, R)$. *This output is not yet the next boundary.* One actual D1 move changes the whole-left word to $4t + 2$ and doubles the right output. The result is the odd boundary with input $4b + 1 = R_{\mathrm{old}}$. Including this alignment move is essential.

12.3 Even boundary with $n = 8b + 2$

Now its even witness is $a = 4b + 1$. Repeated sparse-pop deductions give $\mathrm{Sparse}(b)$. The two inverses supply bounded v, t such that

$$r := R_{\mathrm{old}} = 4v, \quad \mathrm{J}(b, v), \qquad \mathrm{WL} = 16t + 10, \quad \mathrm{T}(b, t).$$

Divide $t = 2x + d$ with $d < 2$. The actual source has $L = 8t + 5$ and $s = 0$. Four carry moves give

Point	State	L	s	R
source	D	$8t+5$	0	r
after D0	C	$4t+2$	1	$2r+1$
after C1	D	$2t+1$	0	$4r+3$
after D0	C	t	1	$8r+7$
after C1	D	x	d	$16r+15$

The last row has $\mathrm{WL} = t$ and $R = 4(4r+3)+3$. It enters the left routine with input b and tail $4r+3$. Its output is A with $\mathrm{WR} = 4r+3$; carry uniqueness identifies its transformed output with v, giving $\mathrm{T}(v, L)$ and $\mathrm{Sparse}(v)$.

Two actual A1 moves clear the two pending low 1s, producing an A row with left half $4L$ and $\mathrm{WR} = r = 4v$. The transform constructor gives $\mathrm{T}(2v, 4L)$ and sparsity gives $\mathrm{Sparse}(2v)$. An A0 move therefore enters the right routine with input $2v$. Its D output is an even boundary with input $4v = r = R_{\mathrm{old}}$.

12.4 Why these cases exhaust the invariant

For an even-clause boundary, $n = 2a$ and $\mathrm{Sparse}(n)$ imply $\mathrm{Sparse}(a)$. Divide $a = 2b + d$, $d < 2$. If $d = 0$, then $n = 4b$. If $d = 1$, the sparse odd-tail lemma forces $b = 2c$ for a bounded witness c, giving $n = 8c + 2$. The odd-clause boundary is already handled directly. Existential elimination discharges all arithmetic witnesses.

The resulting general theorem is

$$\begin{aligned}\mathrm{Total}_o(N) \wedge \mathrm{Steps}_o(N) \wedge h < H \wedge \mathrm{At}_o(N,h) \wedge h_0 = 5 \\ \wedge\, c < H \wedge k < N+1 \wedge \mathrm{At}_o(k,c) \wedge \mathrm{Boundary}(n,c) \quad \rightarrow \\ \exists e < N+1\, \exists d < H\, [k < e \wedge \mathrm{At}_o(e,d) \wedge \mathrm{Boundary}(R(c),d)]. \quad (9)\end{aligned}$$

There is no extra numerical-case assumption here and no new induction in the case combination. Notice that the next input $R(c)$ is below H because it is a coordinate of the old bounded row. We need no separate global growth bound for a sequence of counter values.

13 Putting the pieces together: a worked tour

The preceding sections establish the general argument. This section follows its numerical data more slowly, so that the graph witnesses, machine rows, and induction ranks can be read together. These finite examples illustrate the proved contracts; they do not replace the universal formal proofs.

13.1 What is being carried?

Think of a sparse word as a binary string with no adjacent 1s. Its low end is either 0, or 01 in ordinary, most-significant-first notation. The first case uses $\mathrm{J}(2x, 4x+1)$. In the second case, remove the low 01, process the remaining word, and append two low zeros to the output. For example,

$$5 = (101)_2 = 4 \cdot 1 + 1, \qquad 1 = 4 \cdot 0 + 1.$$

There are two carry blocks. The base word 0 has output 1; restoring the two blocks multiplies the output by 4 twice, giving 16. For $9 = (1001)_2$, there is one carry block. The remaining word is 2, its base output is 5, and restoring the block gives 20.

The graph witnesses describe that process without coding a list of recursive calls. In ordinary mathematics, after k carry blocks the scale is $p = 4^k$ and the consumed suffix is $a = 1+4+\cdots+4^{k-1}$, with $a = 0$ if $k = 0$. The formal proof uses instead the bounded relations $\mathrm{Pow}(p)$ and $p = 3a + 1$, together with $w = p(2z) + a$ and $v = p(4z + 1)$. The variable k in this explanation does not occur in the definition of J. No exponentiation-totality assumption is being smuggled into that bounded formula.

There is a useful elementary identity hidden in its equations:

$$\mathrm{J}(w, v) \longrightarrow \exists a < w + 1\,[v = 2w + a + 1].$$

For clarity, here is the derivation. It uses standard consequences of the ordered semiring axioms, with no induction. This is an explanatory consequence of the definition, not an additional exported kernel theorem.

Line	Equation or conclusion	Reason
1	Assume $\mathrm{J}(w, v)$.	Assumption.
2	Fix its bounded witnesses p, a, z.	Existential elimination in (6).
3	$p = 3a + 1$, $w = 2pz + a$, $v = 4pz + p$.	Conjunction elimination; semiring laws.
4	$2w + a + 1 = 4pz + 3a + 1$.	Substitute w; distribute and associate.
5	$4pz + 3a + 1 = 4pz + p = v$.	Substitute the equations for p and v.
6	$v = 2w + a + 1$.	Transitivity and symmetry of equality.
7	$2w + 1 \le v \le 3w + 1$.	$0 \le a \le w$ and monotonicity of addition.
8	$\exists a < w + 1\,[v = 2w + a + 1]$.	Existential introduction; discharge the fresh witnesses.

Here $\le$ abbreviates $<$ or equality, or equivalently a successor bound. For $w = 5$, the identity says $16 = 2 \cdot 5 + 5 + 1$; for $w = 9$, it says $20 = 2 \cdot 9 + 1 + 1$. It explains why a carry output grows, while remaining bounded by a linear expression in its input.

13.2 Two right-routine instruction cases, without hidden algebra

Start with the B phase of Section 9, scanning 1. Sparsity of $2R + 1$ forces R to be even. Thus there is a bounded u with $R = 2u$. B1 writes 1 and moves right. The actual update equations and quotient/remainder uniqueness give

$$q' = A, \quad L' = 2L + 1, \quad s' = 0, \quad R' = u.$$

In particular, the next scanned bit is proved to be 0; it is not an annotation supplied with the computation. The A-phase equations follow:

$$\begin{aligned} L' + 1 = 2(L + 1) &= 4p(\ell + 1), \\ w = p(4u + 1) + a &= 4pu + 4a + 1, \end{aligned}$$

where $p = 3a + 1$ gives the last equality. Popping a sparse word gives $\mathrm{Sparse}(u)$, and the old bounded witnesses p, a still suffice. The rank falls from $K + 16u + 6$ to $K + 16u + 5$.

Now consider A0. Write the old right half $R = 2r + d$ with $d < 2$. Its actual successor is B with

$L' = 2L + 1$, scanned bit d, and right half r. Set $p' = 4p$ and $a' = 4a + 1$. Then

$$\begin{aligned} p' &= 3a' + 1, \\ L' + 1 &= 8p(\ell + 1) = 2p'(\ell + 1), \\ w &= 4pR + 4a + 1 = p'(2r + d) + a'. \end{aligned}$$

These are exactly the B-phase equations. The existing arithmetic witness-bound lemma gives $a' < w + 1$ and $p' < 3w + 2$. The new whole-right word is the old R, so it is sparse. The rank decreases from $K + 16R + 5$ to $K + 4R + 2$, a positive drop of $12R + 3$.

The local theorem checks every phase in this way. It also eliminates the incompatible alternatives of the full ten-clause Step formula using the source state or scanned bit. Division and the sparse-word lemmas have already been proved; neither of these two instruction checks introduces a new history induction.

13.3 A complete left routine, including its return

Take $w = 1$ and preserved right tail $r = 0$. Its entry has $L = 5, s = 0$ and $R = 3$, so its transform premise is $\mathrm{T}(1, 10)$ and $R = 4r + 3$ holds. The constants are $S = 5$, $K = 40$, and initial rank 56. The actual run is:

Move	State	L	s	R	Phase	Rank
0	D	5	0	3	Search	56
1	C	2	1	7	Carry 1	55
2	D	1	0	15	Carry 2	54
3	C	0	1	31	Carry 3	53
4	D	0	0	63	Search	48
5	C	0	0	127	Exit 1	47
6	E	1	1	63	Exit 2	46
7	B	2	1	31	Exit 3	45
8	A	5	1	15	Exit 4	44
9	A	10	1	7	Clear main	8
10	A	20	1	3	Clear intermediate	5
11	A	40	1	1	Clear main	4
12	A	80	1	0	Clear intermediate	3
13	A	160	0	0	Clear main	2

The common scan witnesses at moves 0–3 are $(p, a, u) = (1, 0, 1)$. Four moves strip the transformed carry block; at move 4 they become $(4, 1, 0)$. They still satisfy $1 = pu + a$ and $p = 3a + 1$. The remaining word 0 takes the exit path. At move 6, E scans 1 and uses E1; the halting E0 instruction is avoided. The exit constructs $V = 4$.

At move 9, the clearing-main witnesses are $p = 4, u = 1, V = 4$. Thus $pu = V$, $\mathrm{T}(1, L)$ with $L = 10$, and $\mathrm{WR} + 1 = 16 = p^2(r + 1)$. At move 10 the intermediate witnesses are $p = 2, u = 1$, so $2pu = 4$ and $\mathrm{WR} + 1 = 8 = 2p^2(r + 1)$. Move 11 returns to main form with $p = 2, u = 2$, left half 40, and $\mathrm{WR} + 1 = 4$. After two further moves the main scale is 1, $u = 4$, the left half is 160 and the restored whole-right word is 0. This is the correct output because $\mathrm{J}(1, 4)$ and $\mathrm{T}(4, 160)$.

Reaching A at move 8 was not enough: the preserved right tail and final left encoding were still unfinished. The clearing phases make those obligations explicit. Also notice that the left tape value grows while the rank decreases. The rank measures unfinished work, not tape magnitude. At move 13 this routine's contract ends; the machine itself continues with A0.

13.4 The first twelve boundaries

Every row below is in state D and satisfies the boundary predicate. The step is the number of ordinary moves from blank. The interpreted input n is obtained by checking the invariant; it is not an extra coordinate supplied on a separate tape.

Step	n	L	s	R	Boundary clause
3	0	0	0	1	even
11	1	1	0	2	odd
25	2	5	0	4	even
39	4	20	0	5	even
51	5	21	0	8	odd
69	8	80	0	9	even
77	9	81	0	10	odd
95	10	85	0	16	even
117	16	320	0	17	even
125	17	321	0	18	odd
139	18	325	0	20	even
153	20	340	0	21	even

At step 25 the even witness is $a = 1$: $n = 2$, WL $= 10$, T$(1, 10)$ and J$(1, 4)$. At step 51 the odd witnesses are $a = 1, t = 10, v = 4$: $n = 5$, WL $= 42 = 4t + 2$ and $R = 8 = 2v$. These different alignments explain why the boundary definition has two clauses.

The next input is the old R. The carry identity above explains why it is larger than n. In the even clause, $R = 2a + \alpha + 1 > 2a = n$ for the consumed suffix α. In the odd clause, $R = 2v = 4a + 2\alpha + 2 > 4a + 1 = n$. This helps one visualize the growing counter. The formal nonhalting proof needs only progress in the *history index*. It does not require a theorem that all sparse numbers are enumerated, or a formula for the displayed step counts.

13.5 Two complete boundary cycles

At step 3 we have $n = 0$, the $n = 4b$ case with $b = t = 0$. Its next eight ordinary moves show exactly why the final alignment move matters:

Step	State	L	s	R	What has happened
3	D	0	0	1	First even boundary, input 0.
4	C	0	0	3	D0 writes 1 and moves left.
5	E	1	1	1	C0 returns right onto a 1.
6	B	2	1	0	E1 moves right; E0 is avoided.
7	A	5	0	0	B1 finishes the four-move prefix.
8	B	11	0	0	A0 enters the right routine: $w = 0, \ell = 5$.
9	C	5	1	0	B0 begins its return.
10	D	2	1	1	C1 gives J$(0, 1)$ and WL $= 5$.
11	D	1	0	2	D1 aligns the odd boundary, input 1.

The row at step 10 is a correct right-routine output, but it is not yet the desired odd boundary. The latter needs WL $= 4 \cdot 0 + 2 = 2$, not 5. D1 changes the whole-left word to 2 and doubles the right half to 2. This is the extra move in the general $n = 4b$ argument.

The next cycle starts with that odd boundary, whose witnesses are $a = 0, t = 0, v = 1$. We now compress only the already proved routines, recording their actual entry and exit rows:

Step	State	L	s	R	Justified segment
11	D	1	0	2	Odd boundary: WL $= 2$, $R = 2 \cdot 1$.
13	D	0	0	11	Two moves D0,C1 give left entry $w = 0$, tail 2.
18	A	10	0	1	Left output: WR $= 2$, J$(0, 1)$, T$(1, 10)$.
19	B	21	1	0	A0 enters the right routine: $w = 1, \ell = 10$.
25	D	5	0	4	Right output: WL $= 10$, J$(1, 4)$; even boundary, input 2.

The odd boundary supplies J$(0, 1)$; the left routine produces some V with J$(0, V)$. Carry uniqueness identifies $V = 1$. One can calculate this particular value directly, but the general odd case needs uniqueness for an arbitrary sparse input.

The counts 8 and 14 are properties of these particular finite examples. The formal progress theorem does not assume a uniform duration. From the proposed ordinary halting history, the routine barriers construct some strictly later boundary index in that very history. That is the input required by the next, greatest-index argument.

14 The last induction: a greatest boundary index

Fix the history codes and parameters. Put

$$P(i) :\Longleftrightarrow \exists n < H \, \exists c < H \, [\mathrm{At}_o(i, c) \wedge \mathrm{Boundary}(n, c)].$$

This is a bounded formula. The final induction formula is

$$M(t): \quad (\exists i < t + 1 \, P(i)) \;\rightarrow\; \exists m < t + 1 \, [P(m) \wedge \forall j < t + 1 \, (P(j) \rightarrow j < m + 1)]. \tag{10}$$

The expression $j < m + 1$ is a language-legal way of writing $j \leq m$. Only t is inducted on. H, N, o and the eight history-code numbers are fixed parameters. The outer existential input n in P is bounded by H, not by a presumed exponential bound.

14.1 Base and successor in detail

At $t = 0$, a witness $i < 1$ must equal 0. Thus nonemptiness gives $P(0)$; choose $m = 0$. Every $j < 1$ already satisfies $j < 0 + 1$.

For the successor, assume $M(t)$ and a witness below $t+2$. Classical excluded middle distinguishes $P(t + 1)$ and $\neg P(t + 1)$.

- If $P(t + 1)$, choose $m = t + 1$. Every scanned index $j < t + 2$ is automatically at most this new maximum.
- If $\neg P(t + 1)$, a nonempty witness below $t + 2$ must already be below $t + 1$. Apply $M(t)$ to obtain an old maximum $m < t + 1$. For a new scanned $j < t + 2$, either $j < t + 1$, when the old maximum applies, or $j = t + 1$, when $P(j)$ contradicts $\neg P(t + 1)$.

All witnesses remain within the displayed bounds. Formula (10) is therefore a valid instance of (3). A maximum principle has been proved, not added as an axiom.

14.2 Fresh variables in the actual natural deduction

Universal introduction must not generalize a variable on which an open assumption depends. The Metamath core enforces a syntactically strict distinct-variable condition. The implementation

proves the scan step using a fresh variable and then applies a separately derived renaming implication. This is logical bookkeeping, not a mathematical assumption that tape values are different. The same issue explains some otherwise surprising renaming lemmas in the machine proofs.

15 The final contradiction as a proof table

Let $\mathcal{H}$ abbreviate the ordinary totality, transition and final-halt conditions, without a blank premise. Named lemmas in the following table are the proved statements from earlier sections. All tuple and predicate abbreviations expand to the given first-order formulas.

Line	Statement or construction	Justification
1	Assume $\mathcal{H} \wedge \exists i < N+1\, P(i)$.	Assumption for negation introduction.
2	$\mathcal{H}$ and $\exists i < N+1\, P(i)$.	Conjunction elimination, 1.
3	There is $m < N+1$ with $P(m)$ and $\forall j < N+1 (P(j) \to j < m+1)$.	Maximum theorem $M(N)$, implication elimination.
4	Fix such a fresh m.	Bounded existential elimination, 3.
5	There are $n < H$ and $c < H$ with $\mathrm{At}_o(m,c) \wedge \mathrm{Boundary}(n,c)$.	Definition of $P(m)$.
6	Fix fresh such n, c.	Successive existential eliminations, 5.
7	There are $e < N+1$ and $d < H$ with $m < e$, $\mathrm{At}_o(e,d)$ and $\mathrm{Boundary}(R(c),d)$.	General progress (9), 2,4,6.
8	Fix fresh such e, d.	Existential elimination, 7.
9	$R(c) < H$.	A coordinate bound from $c < H$.
10	$P(e)$.	Existential introduction with input $R(c)$ and row d; 8,9.
11	$e < m+1$.	Universal elimination in the greatestness assertion, then implication elimination using 8,10.
12	$e < m$ or $e = m$.	Discrete-order below-successor lemma, 11.
13	If $e < m$, then $m < m$; if $e = m$, also $m < m$.	Transitivity using $m < e$, or equality substitution; disjunction elimination.
14	Contradiction.	Order axiom $\neg(m < m)$.
15	$\neg(\mathcal{H} \wedge \exists i < N+1\, P(i))$.	Discharge fresh witnesses and the assumption at 1; negation introduction.

Now add the blank-initial condition. Totality supplies an actual initial row. The blank condition gives its four coordinates as 0 and its bounds give $0 < H$. Advance-before-halt and the three fixed-source instruction lemmas construct the rows at indices 1,2,3, ending at $(3,0,0,1)$. This row satisfies $\mathrm{Boundary}(0,c)$, and hence $P(3)$ with a bounded input 0. Its index is below $N+1$ because every advancement supplied that bound. Line 15 therefore contradicts the original alleged blank-to-halt history. Negation introduction gives exactly (5).

> **Reading note.** If an alleged history has fewer than three moves, the advancement lemma already produces a contradiction at a nonhalting row claimed to be its final halt row. The proof does not quietly assume $N \geq 3$.

16 The five induction constructions, collected

The dependency audit counts proved induction constructions, which may be reused with different parameters. It does not say that full expansion of every lemma use contains only five literal axiom occurrences.

Construction	Formula inducted on, with parameters fixed
Bounded division	$\exists q < i+1\, \exists r < d\, [i = dq + r]$, for fixed $0 < d$.
Right history	Formula (8) with right rank and $R_0 = 20w + 18$.
Left history	Formula (8) with the full scan/clearing rank and $R_0 = 32w + 24$.
Carry uniqueness	Formula (7), with fixed output bound U.
Greatest boundary index	Formula (10), with the fixed bounded predicate P.

Every row is Δ_0 after expanding its abbreviations. The three boundary case calculations and their combination require no additional induction construction. The simplified three-move start uses arithmetic, instruction consequences and history advancement; it does not invoke a routine's history induction.

These are upper bounds for the present proof. No result here says that five inductions are necessary, that weaker arithmetic cannot prove this machine nonhalting, or that this is the smallest possible proof.

17 Finding the corresponding formal proofs

All active mathematical proofs are in the single file `idelta0_machines.mm`. The following suffixes follow `skelet17-ladder-skelet10-`:

Label suffix	What to inspect
`right-counter-entry`, `right-counter-step`	Local rank construction and rank decrease.
`right-window-invariant`	The right-history induction.
`left-counter-entry`, `left-counter-step`	Full left scan and clearing invariant.
`left-window-invariant`	The left-history induction.
`carry-unique-sparse`	Identification of independently supplied carry outputs.
`odd-boundary-cycle`	The odd-clause calculation.
`even-zero-boundary-cycle`	The $n = 4b$ calculation, including its last D1.
`even-one-boundary-cycle`	The $n = 8b+2$ calculation and its two A1 moves.
`boundary-progress`	Exhaustive combination of the three cases.
`boundary-index-maximum`	The explicit greatest-index induction.
`no-halting-history-with-boundary`	The contradiction table of the previous section.
`no-blank-halting-history`	The final theorem with only ordinary blank-history premises.

The three small start-up lemmas have the private labels

```
z10-blank-start-step-1
z10-blank-start-step-2
z10-blank-start-step-3
```

They are instruction consequences, not additional axioms. Metamath can display a full statement, including its mandatory hypotheses and distinct-variable conditions, or the proof of the final theorem with a chosen expansion depth.

17.1 What verification establishes

The kernel checks ordinary Metamath proof steps over the same 93 trusted declarations. Those 93 include syntax, logic and equality infrastructure; they are not 93 extra number-theoretic assumptions. The kernel also checks syntactic certificates that the formula used in bounded induction is genuinely bounded. Such a certificate is a proof-language judgement, not an added arithmetic truth predicate. Python generators only write candidate proofs. A candidate rejected by the kernel is not a proof. Independent statement checks compare the displayed endpoints with separately reconstructed formulas; interface checks preserve earlier theorem conclusions, hypotheses and distinct-variable conditions.

17.2 What the preceding formal rewrite simplified

The start-up argument now consists of the three fixed ordinary transitions from the blank configuration to the first boundary. The previous version used the general right-counter barrier for that segment. The two counter barriers remain necessary for the general boundary calculations.

The rewrite removed 1,174 obsolete private proof nodes and found 1,143 new nodes with existing exact interfaces, allowing reuse before dependency pruning when the hypotheses and distinct-variable conditions permitted it. The explicit new deductions retain 2,249 new nodes, replacing the old endpoint. The net result is 1,074 more expanded proof nodes: 151,838 rather than 150,764. The source grows about 2%. This is a simplification of the mathematical start-up argument and its explanation, not a claim of a smaller expanded proof file. No induction construction or trusted axiom is added by this revision.

All 270 published interfaces are retained, including the exact final statement. The full kernel check, exact-interface comparison and actual proof-dependency audit are the appropriate evidence for this change.

The large source file includes the earlier machine ladder, expanded first-order history formulas, and many syntax and equality proof nodes. Its size is not the number of distinct mathematical ideas in the nonhalting argument. The useful abstraction is: two local routines, three boundary cases, one greatest-index argument, and the ordinary start.

18 Scope of this draft and further reading

This second explanatory revision adds the carry calculation, two local instruction derivations, a complete left-routine trace, twelve concrete boundaries, two boundary-cycle walkthroughs and a halting-barrier proof table. It leaves the canonical Metamath file byte-for-byte unchanged.

This draft gives explicit definitions, the rank and boundary invariants, the five induction formulas, worked examples, and the final natural- deduction structure. It leaves many elementary arithmetic details inside named verified lemmas. In particular, it is not an independently expanded line-by-line printing of the entire Metamath dependency closure.

The formal result is about Skelet #10. It does not prove a Busy Beaver value, the soundness of an arbitrary new decider, a lower bound on the required theory, or Skelet #17 nonhalting. The current checkpoint's verification reports give the exact canonical source hash and the measured effect of the start-up simplification.

References

[1] Project formal source, `idelta0_machines.mm`, with theorem exports and verification reports in the accompanying checkpoint. The theorem labels in this document identify the relevant formal statements.

[2] Project generators: `ladder_skelet10_right.py`, `ladder_skelet10_full_left.py`, `ladder_skelet10_carry_unique.py`, the boundary-cycle modules, `ladder_skelet10_boundary_maximum.py`, and `ladder_skelet10_blank_endpoint.py`. The source is evidence for the proof's construction; the Metamath file is what the kernel verifies.

[3] Norman D. Megill and David A. Wheeler, *Metamath: A Computer Language for Mathematical Proofs*, 2019. https://us.metamath.org/downloads/metamath.pdf. For the proof language, substitution, hypotheses and verification; this project supplies its own arithmetic core rather than importing ZFC.